Hotel Management System — Extension Practice Questions with Code Snippets & Comments

---

1. Implement Room Class and Room Management

Question:
Create a `Room` class with attributes: `roomNumber`, `roomType` (Single, Double, Suite), `price`, and `availability`. Add menu options to add rooms and print room availability.

Answer:

java
CopyEdit

```
// Room.java
package com.mycompany.hotelmanagementsystem;

public class Room {
    // Room number identifier
    private int roomNumber;

    // Type of room: Single, Double, Suite, etc.
    private String roomType;

    // Price per night for this room
    private double price;

    // Indicates if room is available or occupied
    private boolean available;

    // Constructor initializes all fields; room is available by
default
    public Room(int roomNumber, String roomType, double price) {
        this.roomNumber = roomNumber;
        this.roomType = roomType;
```

```
        this.price = price;
        this.available = true; // Newly added rooms are available
    }

    // Getter for room number
    public int getRoomNumber() {
        return roomNumber;
    }

    // Getter for room type
    public String getRoomType() {
        return roomType;
    }

    // Getter for price
    public double getPrice() {
        return price;
    }

    // Check if the room is available
    public boolean isAvailable() {
        return available;
    }

    // Set availability (true for available, false for occupied)
    public void setAvailable(boolean available) {
        this.available = available;
    }

    // String representation for printing room info
    @Override
    public String toString() {
        return "Room " + roomNumber + " (" + roomType + "), Price: " +
price + ", Available: " + available;
    }
}
```

java
CopyEdit

```
// WestminsterHotelManager.java (add to class fields)
private ArrayList<Room> roomList = new ArrayList<>(); // List to store
all rooms

// Add menu options in runMenu()
System.out.println("To Add a new room, press 5");
System.out.println("To View room availability, press 6");

// In switch-case of runMenu(), add cases
switch (choice) {
    // existing cases...
    case 5:
        addRoom();
        break;
    case 6:
        printRoomAvailability();
        break;
}

// Method to add a new room
public void addRoom() {
    Scanner s = new Scanner(System.in);

    System.out.println("Enter room number:");
    int number = s.nextInt();
    s.nextLine(); // consume newline

    System.out.println("Enter room type (Single, Double, Suite):");
    String type = s.nextLine();

    System.out.println("Enter room price:");
    double price = s.nextDouble();
    s.nextLine();

    Room room = new Room(number, type, price);
    roomList.add(room); // Add to list

    System.out.println("Room added successfully.");
```

```
}

// Method to print all rooms with availability status
public void printRoomAvailability() {
    if (roomList.isEmpty()) {
        System.out.println("No rooms added.");
    } else {
        // Iterate over all rooms and print details
        for (Room room : roomList) {
            System.out.println(room.toString());
        }
    }
}
```

---

2. Implement Guest Booking with Room Availability Check

Question:
When adding a guest, assign them an available room of their preferred type automatically. If no rooms are available, notify the user.

Answer:

java
CopyEdit

```
// Modify addGuest() method to assign room based on preference
@Override
public void addGuest() {
    Scanner s = new Scanner(System.in);

    System.out.println("Enter the first name:");
    String name = s.nextLine();

    System.out.println("Enter the last name:");
    String surname = s.nextLine();

    System.out.println("Enter preferred room type (Single, Double,
Suite):");
    String preferredType = s.nextLine();
```

```
    // Find an available room matching the preferred type
    Room availableRoom = null;
    for (Room room : roomList) {
        if (room.isAvailable() &&
room.getRoomType().equalsIgnoreCase(preferredType)) {
            availableRoom = room;
            break; // assign first available matching room
        }
    }

    // If no rooms found, notify and return
    if (availableRoom == null) {
        System.out.println("No available rooms of type " +
preferredType);
        return;
    }

    System.out.println("Enter the number of nights stayed:");
    int nightsStayed = s.nextInt();
    s.nextLine();

    // Create new guest object
    Guest guest = new Guest(name, surname);

    // Assign room number to guest
    guest.setRoomNumber(availableRoom.getRoomNumber());

    // Set number of nights staying
    guest.setNightsStayed(nightsStayed);

    // Add guest to guest list
    hotelGuestList.add(guest);

    // Mark room as occupied
    availableRoom.setAvailable(false);
```

```
    System.out.println("Guest added successfully to room " +
availableRoom.getRoomNumber());
}
```

---

3. Implement Guest Check-in and Check-out Process

Question:
Add options to check-in guests (record current date), and check-out guests (calculate bill and free room).

Answer:

java
CopyEdit

```
// Guest.java (extend Guest class)
import java.time.LocalDate;

public class Guest {
    // Existing fields
    private LocalDate checkInDate; // Date of check-in
    private boolean checkedIn; // Status if checked in

    // Existing constructor and methods ...

    // Mark guest as checked in and record current date
    public void checkIn() {
        this.checkInDate = LocalDate.now();
        this.checkedIn = true;
    }

    // Mark guest as checked out
    public void checkOut() {
        this.checkedIn = false;
    }

    public LocalDate getCheckInDate() {
        return checkInDate;
    }
```

```
    public boolean isCheckedIn() {
        return checkedIn;
    }

    // Extend toString to include check-in status
    @Override
    public String toString() {
        return super.toString() + ", Checked In: " + checkedIn +
               (checkedIn ? ", Check-in date: " + checkInDate : "");
    }
}
```

java
CopyEdit

```
// WestminsterHotelManager.java - add menu options
System.out.println("To Check-in a guest, press 7");
System.out.println("To Check-out a guest, press 8");

switch(choice) {
    // existing cases...
    case 7:
        checkInGuest();
        break;
    case 8:
        checkOutGuest();
        break;
}

// Method to check-in a guest
public void checkInGuest() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter guest's first name to check-in:");
    String name = s.nextLine();

    for (Guest guest : hotelGuestList) {
        // Find guest not already checked in
        if (guest.getName().equalsIgnoreCase(name) &&
!guest.isCheckedIn()) {
```

```
            guest.checkIn();
            System.out.println("Guest " + name + " checked in.");
            return;
        }
    }
    System.out.println("Guest not found or already checked in.");
}

// Method to check-out a guest and calculate bill
public void checkOutGuest() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter guest's first name to check-out:");
    String name = s.nextLine();

    for (Guest guest : hotelGuestList) {
        // Find guest currently checked in
        if (guest.getName().equalsIgnoreCase(name) &&
guest.isCheckedIn()) {
            guest.checkOut();

            // Find the guest's room
            Room guestRoom = null;
            for (Room room : roomList) {
                if (room.getRoomNumber() == guest.getRoomNumber()) {
                    guestRoom = room;
                    break;
                }
            }

            if (guestRoom != null) {
                // Mark room available after checkout
                guestRoom.setAvailable(true);

                // Calculate total bill based on nights stayed and
room price
                double bill = guest.getNightsStayed() *
guestRoom.getPrice();
```

```
                System.out.println("Guest checked out. Total bill: $"
+ bill);
            }
            return;
        }
    }
    System.out.println("Guest not found or not currently checked
in.");
}
```

---

4. Implement Booking Cancellation

Question:
Add a menu option to cancel a guest's booking by guest name.

Answer:

java
CopyEdit

```
// WestminsterHotelManager.java - add menu option
System.out.println("To Cancel booking, press 9");

switch(choice) {
    // existing cases ...
    case 9:
        cancelBooking();
        break;
}

// Method to cancel booking and free room
public void cancelBooking() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter guest's first name to cancel booking:");
    String name = s.nextLine();

    for (int i = 0; i < hotelGuestList.size(); i++) {
        Guest guest = hotelGuestList.get(i);
        if (guest.getName().equalsIgnoreCase(name)) {
```

```
            // Free the room occupied by guest
            for (Room room : roomList) {
                if (room.getRoomNumber() == guest.getRoomNumber()) {
                    room.setAvailable(true);
                }
            }

            // Remove guest from list
            hotelGuestList.remove(i);
            System.out.println("Booking for guest " + name + "
cancelled.");
            return;
        }
    }
    System.out.println("Guest booking not found.");
}
```

---

5. Extend Guest List Printing

Question:
Modify guest list printing to show room details (room number and type) and booking status.

Answer:

java
CopyEdit

```
// WestminsterHotelManager.java - print guest list with room info
public void printGuestList() {
    if (hotelGuestList.isEmpty()) {
        System.out.println("No guests found.");
    } else {
        // Iterate over guests
        for (Guest guest : hotelGuestList) {
            // Prepare room info string
            String roomInfo = "Room: " + guest.getRoomNumber();
            Room room = null;

            // Find room object matching guest's room number
```

```
            for (Room r : roomList) {
                if (r.getRoomNumber() == guest.getRoomNumber()) {
                    room = r;
                    break;
                }
            }

            // Add room type if found
            if (room != null) {
                roomInfo += " (" + room.getRoomType() + ")";
            }

            // Print guest details + room info
            System.out.println(guest.toString() + ", " + roomInfo);
        }
    }
}

// Add menu option to print guest list
System.out.println("To Print guest list, press 10");

switch(choice) {
    // existing cases...
    case 10:
        printGuestList();
        break;
}
```

---

6. Add Staff Salary Attribute and Management

Question:
Add a `salary` attribute to `HotelStaff`. Implement menu options to set a staff member's salary and view all salaries.

Answer:

java
CopyEdit

```
// HotelStaff.java - add salary attribute
public class HotelStaff {
    // Existing fields...

    private double salary; // salary attribute

    // Getter for salary
    public double getSalary() {
        return salary;
    }

    // Setter for salary
    public void setSalary(double salary) {
        this.salary = salary;
    }

    // Modify toString() to include salary info
    @Override
    public String toString() {
        return super.toString() + ", Salary: $" + salary;
    }
}
```

java
CopyEdit

```
// WestminsterHotelManager.java - add menu options
System.out.println("To Set salary for staff, press 11");
System.out.println("To View all staff salaries, press 12");

switch(choice) {
    // existing cases...
    case 11:
        setStaffSalary();
        break;
    case 12:
        printStaffSalaries();
        break;
}
```

```
// Method to set salary for a staff member by ID
public void setStaffSalary() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter staff ID:");
    String id = s.nextLine();

    for (HotelStaff staff : hotelStaffList) {
        if (staff.getStaffID().equals(id)) {
            System.out.println("Enter salary amount:");
            double sal = s.nextDouble();
            s.nextLine();
            staff.setSalary(sal);
            System.out.println("Salary updated.");
            return;
        }
    }
    System.out.println("Staff ID not found.");
}

// Method to print all staff salaries
public void printStaffSalaries() {
    if (hotelStaffList.isEmpty()) {
        System.out.println("No staff found.");
    } else {
        for (HotelStaff staff : hotelStaffList) {
            System.out.println(staff.getName() + " " +
staff.getSurname() + " - Salary: $" + staff.getSalary());
        }
    }
}
```

---

7. Add Search and Filter Features

Question:
Implement menu options to search guests by name or room number, and filter staff by role (Manager or HouseKeeper).

Answer:

java
CopyEdit

```
// WestminsterHotelManager.java - add menu options
System.out.println("To Search guest by name, press 13");
System.out.println("To Search guest by room number, press 14");
System.out.println("To Filter staff by role, press 15");

switch(choice) {
    // existing cases...
    case 13:
        searchGuestByName();
        break;
    case 14:
        searchGuestByRoom();
        break;
    case 15:
        filterStaffByRole();
        break;
}

// Search guests by first name
public void searchGuestByName() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter guest name to search:");
    String name = s.nextLine();

    for (Guest guest : hotelGuestList) {
        if (guest.getName().equalsIgnoreCase(name)) {
            System.out.println(guest.toString());
        }
```

```
    }
}

// Search guests by room number
public void searchGuestByRoom() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter room number to search:");
    int roomNum = s.nextInt();
    s.nextLine();

    for (Guest guest : hotelGuestList) {
        if (guest.getRoomNumber() == roomNum) {
            System.out.println(guest.toString());
        }
    }
}

// Filter staff by role: Manager or HouseKeeper
public void filterStaffByRole() {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter role to filter (Manager or
HouseKeeper):");
    String role = s.nextLine();

    for (HotelStaff staff : hotelStaffList) {
        // Check if staff is Manager and role input is Manager
        if (role.equalsIgnoreCase("Manager") && staff instanceof
Manager) {
            System.out.println(staff.toString());
        }
        // Check if staff is HouseKeeper and role input is HouseKeeper
        else if (role.equalsIgnoreCase("HouseKeeper") && staff
instanceof HouseKeeper) {
            System.out.println(staff.toString());
        }
    }
}
```

Summary Table of Extension Questions

No.	Question Summary	Method(s) / Class Modified	Menu Option Number
1	Implement Room class & add room management	Room.java, addRoom(), printRoomAvailability()	5 (Add room), 6 (View rooms)
2	Assign available room automatically to guest	Modify addGuest()	Existing (extend)
3	Guest check-in and check-out process	Guest.java, checkInGuest(), checkOutGuest()	7 (Check-in), 8 (Check-out)
4	Cancel guest booking	cancelBooking()	9
5	Print guest list with room details	printGuestList()	10
6	Add salary attribute and manage staff salaries	HotelStaff.java, setStaffSalary(), printStaffSalaries()	11 (Set salary), 12 (View salaries)

7	Search guests by name/room and filter staff by role	searchGuestByName(), searchGuestByRoom(), filterStaffByRole()	13, 14, 15

Short Answer Questions with Code Snippets and Comments

---

1. Define a `Room` class with room number and availability status.

java
CopyEdit

```
public class Room {
    private int roomNumber;
    private boolean available;

    // Constructor sets room number and marks as available by default
    public Room(int roomNumber) {
        this.roomNumber = roomNumber;
        this.available = true; // Room is available when created
    }

    // Getter for room number
    public int getRoomNumber() { return roomNumber; }

    // Getter for availability status
    public boolean isAvailable() { return available; }

    // Setter to update availability
    public void setAvailable(boolean available) { this.available =
available; }
}
```

---

2. Add a new Room object to a list of rooms.

java
CopyEdit

```
// Create list to hold Room objects
ArrayList<Room> rooms = new ArrayList<>();

// Add a new room with number 101
```

```
rooms.add(new Room(101));
```

---

3. Check if any "Single" room is available.

java
CopyEdit

```
for (Room r : rooms) {
    // Assuming getRoomType() returns the type of room, e.g. "Single"
    if (r.isAvailable() && r.getRoomType().equalsIgnoreCase("Single"))
{
        System.out.println("Room " + r.getRoomNumber() + " is
available.");
        break; // Stop after finding one available room
    }
}
```

---

4. Mark a room as occupied.

java
CopyEdit

```
// Set the room availability to false (occupied)
room.setAvailable(false);
```

---

5. Override `toString()` method in a class to customize printing.

java
CopyEdit

```
@Override
public String toString() {
    // Return string describing room number and availability
    return "Room " + roomNumber + " - Available: " + available;
}
```

---

6. Read integer and string input from user.

java

CopyEdit

```
Scanner sc = new Scanner(System.in);

int number = sc.nextInt();    // Read integer input
sc.nextLine();                // Consume leftover newline

String text = sc.nextLine();  // Read string input
```

---

7. Mark a guest as checked-in and set check-in date.

java
CopyEdit

```
// Mark guest as checked in
guest.setCheckedIn(true);

// Set check-in date to current date
guest.setCheckInDate(LocalDate.now());
```

---

8. Calculate total bill based on nights stayed and room price.

java
CopyEdit

```
// Multiply number of nights stayed by price per night
double bill = guest.getNightsStayed() * room.getPrice();

System.out.println("Total bill: $" + bill);
```

---

9. Remove a guest by name from the guest list.

java
CopyEdit

```
for (int i = 0; i < guests.size(); i++) {
    if (guests.get(i).getName().equalsIgnoreCase(name)) {
        guests.remove(i);  // Remove guest at index i
        break;             // Exit loop after removal
    }
}
```

---

10. Print details of all staff members who are Managers.

java
CopyEdit

```
for (HotelStaff staff : staffList) {
    if (staff instanceof Manager) {
        System.out.println(staff.toString());  // Print manager info
    }
}
```

---

11. Add a salary attribute to a staff class with getter and setter.

java
CopyEdit

```
// Field for staff salary
private double salary;

// Getter for salary
public double getSalary() { return salary; }

// Setter for salary
public void setSalary(double salary) { this.salary = salary; }
```

---

12. Print all guests' details.

java
CopyEdit

```
for (Guest guest : guestList) {
    System.out.println(guest.toString());  // Print guest details
}
```

---

13. Search guest by room number.

java
CopyEdit

```
int searchRoom = 101;

for (Guest guest : guestList) {
    if (guest.getRoomNumber() == searchRoom) {
        System.out.println(guest.toString());  // Print matching guest
info
    }
}
```

---

14. Add a new menu option in CLI.

java
CopyEdit

```
System.out.println("Press 1 to add staff");
int choice = scanner.nextInt();

switch(choice) {
    case 1:
        addStaff();  // Call method to add staff member
        break;
    // Add other cases as needed
}
```

---

15. Mark a room available after guest checkout.

java
CopyEdit

```
// Set room as available again after guest leaves
room.setAvailable(true);
```

#	Question	Code Snippet Summary
1	Define a `Room` class with room number & availability	Class with `roomNumber`, `available` boolean, getters/setters
2	Add a new Room object to list	`rooms.add(new Room(101));`
3	Check if any "Single" room is available	Loop `rooms` and check `isAvailable()` and `getRoomType()`
4	Mark a room as occupied	`room.setAvailable(false);`
5	Override `toString()` in a class	Return custom string with room info
6	Read integer and string input from user	Use `Scanner.nextInt()`, `nextLine()` to read inputs
7	Mark guest checked-in and set check-in date	`guest.setCheckedIn(true);` `guest.setCheckInDate(LocalDate.now());`
8	Calculate total bill from nights stayed and price	Multiply `nightsStayed * room.getPrice()`
9	Remove guest by name from guest list	Loop `guestList`, match name, `remove(i)`
10	Print all staff who are Managers	Loop staff list, check `instanceof Manager`, print
11	Add salary attribute with getter/setter	Add `private double salary;` with `getSalary()`, `setSalary()`
12	Print all guests' details	Loop guest list, print `toString()`
13	Search guest by room number	Loop guests, check `getRoomNumber()`, print match
14	Add new menu option in CLI	Add print statement and `case` in switch for menu
15	Mark room available after guest checkout	`room.setAvailable(true);`

OOP Java Lab Exam Questions - Hotel Management System

BASIC LEVEL QUESTIONS

1. Inheritance Questions

Q1.1: Add a new staff type called "Receptionist" that extends HotelStaff with an additional attribute "languagesSpoken" (ArrayList<String>).

Answer Location: Create new file `Receptionist.java`

```
public class Receptionist extends HotelStaff {
    private ArrayList<String> languagesSpoken;

    public Receptionist(String name, String surname) {
        super(name, surname);
        languagesSpoken = new ArrayList<>();
    }

    public void addLanguage(String language) {
        languagesSpoken.add(language);
    }

    public ArrayList<String> getLanguagesSpoken() {
        return languagesSpoken;
    }

    @Override
    public String toString() {
        return super.toString() + " Receptionist - Languages: " + languagesSpoken;
    }
}
```

Modifications needed in WestminsterHotelManager.java:

- In `addHotelStaff()` method, add case 3 for Receptionist
- Update menu to show "Press 3 if you want to add a Receptionist"

Q1.2: Make the Guest class extend a new abstract class called "Person" along with HotelStaff.

Answer: Create new `Person.java` file:

```
public abstract class Person {
    protected String name;
    protected String surname;

    public Person(String name, String surname) {
        this.name = name;
        this.surname = surname;
    }

    // Common getters and setters
    public String getName() { return name; }
    public String getSurname() { return surname; }
    public void setName(String name) { this.name = name; }
    public void setSurname(String surname) { this.surname = surname; }
}
```

Modify HotelStaff.java: Change `extends` to `extends Person` and remove name/surname attributes **Modify Guest.java:**Change to `extends Person` and remove name/surname attributes

---

2. Polymorphism Questions

Q2.1: Create a method in WestminsterHotelManager called `displayStaffInfo()` that uses polymorphism to display different information for each staff type.

Answer Location: Add to `WestminsterHotelManager.java`

```
public void displayStaffInfo() {
    for (HotelStaff staff : hotelStaffList) {
        System.out.println(staff.toString()); // Polymorphic call

        if (staff instanceof Manager) {
            Manager manager = (Manager) staff;
            System.out.println("License: " + manager.getLicenseNumber());
        } else if (staff instanceof HouseKeeper) {
            HouseKeeper housekeeper = (HouseKeeper) staff;
            System.out.println("Experience: " + housekeeper.getYearsOfExperience() + " years");
        }
    }
}
```

Q2.2: Add an abstract method `calculateSalary()` to HotelStaff and implement it differently in Manager and HouseKeeper classes.

Answer Location: In HotelStaff.java: Add `public abstract double calculateSalary();` **In Manager.java:** Add implementation:

```
@Override
public double calculateSalary() {
    return 50000.0; // Base manager salary
}
```

In HouseKeeper.java: Add implementation:

```
@Override
public double calculateSalary() {
    return 25000.0 + (yearsOfExperience * 1000); // Base + experience bonus
}
```

INTERMEDIATE LEVEL QUESTIONS

3. Encapsulation Questions

Q3.1: Add validation to the Manager class so that license number must be exactly 8 characters and contain only alphanumeric characters.

Answer Location: Modify `setLicenseNumber` in `Manager.java`

```
public void setLicenseNumber(String licenseNumber) {
    if (licenseNumber != null && licenseNumber.length() == 8 &&
        licenseNumber.matches("[A-Za-z0-9]+")) {
        this.licenseNumber = licenseNumber;
    } else {
        throw new IllegalArgumentException("License number must be 8 alphanumeric
characters");
    }
}
```

Q3.2: Make the hotelStaffList in WestminsterHotelManager private and add a method to get a copy of the list instead of direct access.

Answer Location: In `WestminsterHotelManager.java`

```
public ArrayList<HotelStaff> getStaffListCopy() {
    return new ArrayList<>(hotelStaffList);
}
```

---

4. Interface Implementation Questions

Q4.1: Create an interface called "Payable" with method `calculatePay(int hoursWorked)` and implement it in all staff classes.

Answer: Create `Payable.java`:

```
public interface Payable {
    double calculatePay(int hoursWorked);
}
```

Modify staff classes: Add `implements Payable` and implement the method

Q4.2: Add a new method `removeHotelStaff(String staffID)` to the HotelManager interface and implement it.

Answer Location: In HotelManager.java: Add `void removeHotelStaff(String staffID);` **In WestminsterHotelManager.java:** Implement:

```
@Override
public void removeHotelStaff(String staffID) {
    hotelStaffList.removeIf(staff -> staff.getStaffID().equals(staffID));
}
```

ADVANCED LEVEL QUESTIONS

5. Exception Handling Questions

Q5.1: Add custom exception handling for when trying to add staff beyond the limit.

Answer: Create `StaffLimitExceededException.java`:

```
public class StaffLimitExceededException extends Exception {
    public StaffLimitExceededException(String message) {
        super(message);
    }
}
```

Modify addStaffToList in WestminsterHotelManager.java:

```
public void addStaffToList(HotelStaff staff) throws StaffLimitExceededException {
    if (this.hotelStaffList.size() >= staffLimit) {
        throw new StaffLimitExceededException("Cannot add more staff. Limit reached: " +
staffLimit);
    }
    hotelStaffList.add(staff);
}
```

Q5.2: Add try-catch blocks in the addHotelStaff method to handle invalid input.

Answer Location: Wrap input validation in `addHotelStaff()` method with try-catch blocks

---

6. Collections and Generics Questions

Q6.1: Modify the code to use HashMap instead of ArrayList to store staff members, using staffID as the key.

Answer Location: In `WestminsterHotelManager.java`

- Change `ArrayList<HotelStaff> hotelStaffList` to `HashMap<String, HotelStaff> hotelStaffMap`
- Update all methods accordingly

Q6.2: Add a method to sort staff members by their date of birth.

Answer Location: Add to `WestminsterHotelManager.java`

```
public void sortStaffByDateOfBirth() {
    hotelStaffList.sort((s1, s2) -> s1.getDateOfBirth().compareTo(s2.getDateOfBirth()));
}
```

---

7. GUI Integration Questions

Q7.1: Add functionality to the "Info" button in HotelTableGUI to show detailed information about selected staff member.

Answer Location: In `HotelTableGUI.java` constructor, add ActionListener:

```
button.addActionListener(e -> {
    int selectedRow = myTable.getSelectedRow();
    if (selectedRow >= 0) {
        HotelStaff selectedStaff = list.get(selectedRow);
        JOptionPane.showMessageDialog(this, selectedStaff.toString());
    }
});
```

Q7.2: Add a new column to the table showing the staff member's calculated salary.

Answer Location: In HotelTableModel.java:

- Update columnNames array to include "Salary"
- Modify getColumnCount() and getValueAt() methods

COMPLEX DESIGN PATTERN QUESTIONS

8. Factory Pattern Questions

Q8.1: Implement a Factory pattern to create different types of staff members.

Answer: Create `StaffFactory.java`:

```
public class StaffFactory {
    public static HotelStaff createStaff(String type, String name, String surname) {
        switch (type.toLowerCase()) {
            case "manager":
                return new Manager(name, surname);
            case "housekeeper":
                return new HouseKeeper(name, surname);
            default:
                throw new IllegalArgumentException("Unknown staff type: " + type);
        }
    }
}
```

---

9. Observer Pattern Questions

Q9.1: Implement Observer pattern where GUI is notified when new staff is added.

Answer: Create observer interface and implement in GUI classes

---

DATA STRUCTURE AND ALGORITHM QUESTIONS

10. Search and Filter Questions

Q10.1: Add a method to search for staff members by name (case-insensitive).

Answer Location: Add to `WestminsterHotelManager.java`

```
public ArrayList<HotelStaff> searchStaffByName(String searchName) {
    ArrayList<HotelStaff> results = new ArrayList<>();
    for (HotelStaff staff : hotelStaffList) {
        if (staff.getName().toLowerCase().contains(searchName.toLowerCase()) ||
            staff.getSurname().toLowerCase().contains(searchName.toLowerCase())) {
            results.add(staff);
        }
    }
    return results;
}
```

Q10.2: Add filtering functionality to show only managers or only housekeepers.

Answer Location: Add methods in `WestminsterHotelManager.java`

```
public ArrayList<Manager> getAllManagers() {
    return hotelStaffList.stream()
        .filter(staff -> staff instanceof Manager)
        .map(staff -> (Manager) staff)
        .collect(Collectors.toCollection(ArrayList::new));
}
```

FILE I/O AND SERIALIZATION QUESTIONS

11. Data Persistence Questions

Q11.1: Add functionality to save and load hotel data to/from a file.

Answer: Add methods in `WestminsterHotelManager.java` using ObjectOutputStream/ObjectInputStream

---

Q11.2: Implement CSV export functionality for staff data.

Answer: Create method to write staff data to CSV format

VALIDATION AND BUSINESS LOGIC QUESTIONS

12. Data Validation Questions

Q12.1: Add validation to ensure staff ID is unique when adding new staff.

Answer Location: In `addHotelStaff()` method, add check:

```
private boolean isStaffIDUnique(String staffID) {
    return hotelStaffList.stream().noneMatch(staff -> staff.getStaffID().equals(staffID));
}
```

Q12.2: Add age validation (staff must be at least 18 years old).

Answer Location: Add validation in date input section of `addHotelStaff()`

TESTING AND DEBUGGING QUESTIONS

13. Code Quality Questions

Q13.1: Add toString() method to Guest class (currently missing proper implementation).

Answer Location: Fix `Guest.java` toString method:

```
@Override
public String toString() {
    return name + " " + surname + " - Room: " + roomNumber + ", Nights: " + nightsStayed;
}
```

Q13.2: Fix the potential null pointer exception in getStringDate() method.

Answer Location: In `HotelStaff.java`:

```
public String getStringDate() {
    if (dateOfBirth == null) {
        return "Date not set";
    }
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    return dateOfBirth.format(formatter);
}
```

🔍 Sample Exam Questions & Code Change Locations

✅ 1. Add Logging for Staff and Guest Additions

🧠 **Question:** Add a logging system to print messages when staff or guests are added. Each log should contain the name, role, and timestamp.

📂 **Files:** - Logger.java (new) - WestminsterHotelManager.java

```
// Logger.java
public class Logger {
    public static void log(String message) {
        System.out.println("[" + java.time.LocalDateTime.now() + "] " + message);
    }
}
```

In WestminsterHotelManager.java → inside addHotelStaff():

```
Logger.log("Added Manager: " + name + " " + surname);
Logger.log("Added Housekeeper: " + name + " " + surname);
```

In WestminsterHotelManager.java → inside addGuest():

```
Logger.log("Added Guest: " + name + " " + surname);
```

✅ 2. Add a New Role: Receptionist

🧠 **Question:** Create a new staff role Receptionist with a languageSpoken property. Integrate it into the system.

📂 **Files:** - Receptionist.java (new) - WestminsterHotelManager.java - HotelTableModel.java

```
// Receptionist.java
public class Receptionist extends HotelStaff {
    private String languageSpoken;

    public Receptionist(String name, String surname) {
        super(name, surname);
    }

    public void setLanguageSpoken(String languageSpoken) {
        this.languageSpoken = languageSpoken;
    }

    public String getLanguageSpoken() {
        return languageSpoken;
    }

    @Override
    public String toString() {
        return super.toString() + " Receptionist - Language: " + languageSpoken;
    }
}
```

Update CLI in WestminsterHotelManager.java → add case 3:

```
case 3:
    System.out.println("Enter language spoken:");
    String lang = s.nextLine();
    Receptionist rec = new Receptionist(name, surname);
    rec.setLanguageSpoken(lang);
    rec.setDateOfBirth(date);
    rec.setStaffID(staffID);
    this.addStaffToList(rec);
    Logger.log("Added Receptionist: " + name + " " + surname);
    break;
```

Update HotelTableModel.java → inside getValueAt():

```
else if (list.get(rowIndex) instanceof Receptionist) {
    temp = "Receptionist";
}
```

3. Save Staff List to File

🧠 **Question:** Add a method that writes the list of hotel staff to staff.txt.

📂 **File:** WestminsterHotelManager.java

```
public void saveStaffToFile() {
    try (PrintWriter writer = new PrintWriter("staff.txt")) {
        for (HotelStaff member : hotelStaffList) {
            writer.println(member.toString());
        }
        Logger.log("Staff list saved to staff.txt");
    } catch (IOException e) {
        System.out.println("Error saving file: " + e.getMessage());
    }
}
```

Call this method after adding staff if needed.

✅ 4. Handle Invalid Date Input with Custom Exception

🧠 **Question:** Add a custom exception InvalidDateFormatException to handle wrong date formats.

📂 **Files:** - InvalidDateFormatException.java (new) - WestminsterHotelManager.java

```
// InvalidDateFormatException.java
public class InvalidDateFormatException extends Exception {
    public InvalidDateFormatException(String msg) {
        super(msg);
    }
}
```

In WestminsterHotelManager.java → inside date parsing block:

```
while (!parsingSucceeds) {
    dob = s.nextLine();
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    try {
        date = LocalDate.parse(dob, formatter);
        parsingSucceeds = true;
    } catch (DateTimeParseException e) {
        try {
            throw new InvalidDateFormatException("Date format must be dd/MM/yyyy");
        } catch (InvalidDateFormatException ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

✅ 5. Guest Reporter Thread (Concurrency)

🧠 **Question:** Create a background thread that logs the total guest count every 5 seconds.

📂 **Files:** - GuestReporter.java (new) - HotelManagementSystem.java

```
// GuestReporter.java
public class GuestReporter implements Runnable {
    private final ArrayList<Guest> guestList;

    public GuestReporter(ArrayList<Guest> guestList) {
        this.guestList = guestList;
    }

    public void run() {
        while (true) {
            Logger.log("Total guests: " + guestList.size());
            try {
                Thread.sleep(5000);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}
```

In HotelManagementSystem.java → start the thread:

```
GuestReporter reporter = new GuestReporter(((WestminsterHotelManager)
hotelManager).getHotelGuestList());
new Thread(reporter).start();
```

In WestminsterHotelManager.java → add getter:

```
public ArrayList<Guest> getHotelGuestList() {
    return hotelGuestList;
}
```